

Eventim Disability Integration

Barrierefreie Abwicklung von Ticketbestellungen & Nachteilsausgleichsanträgen

Version 1.0 – 23. Juli 2025

Eventim Disability Integration

Dieses Repository enthält eine Next.js Anwendung samt Express Backend, mit der Eventim eine barrierefreie Abwicklung von Ticketbestellungen und eine Verwaltung von Nachteilsausgleichsanträgen ermöglicht. Das Projekt gliedert sich in ein Frontend und einen Node.js Server im Unterordner `eventim-disability-integration`.

Inhaltsverzeichnis

- [Setup](#)
- [Architektur und eingesetzte Technologien](#)
- [Datenbank](#)
- [Backend-Endpunkte](#)
- [Seitenübersicht](#)
- [Rollen](#)
- [Komponenten und Hooks](#)
- [Weiterführende Hinweise](#)
- [Projektstruktur](#)
- [Security & Fehlertoleranz](#)
- [Nächste Schritte](#)

Setup

1. **Repository klonen** und in das Projekt wechseln:

```
git clone <repo-url>
cd EventimDisabilityIntegration/eventim-disability-
integration
```

2. **Abhängigkeiten installieren:**

```
npm install
```

3. Datenbankzugang konfigurieren:

Legen Sie im Verzeichnis `server` eine Datei `credentials.json` an. Beispiel:

```
{
  "host": "152.53.119.113",
  "port": 5433,
  "user": "postgres",
  "password": "example",
  "database": "db1",
  "sessionSecret": "example-secret"
}
```

4. Datenbank anlegen:

- PostgreSQL muss installiert sein.
- Das Schema finden Sie in `server/backup_script.sql` und kann z.B. mit `psql` eingespielt werden:

```
psql -U dbuser -d eventim -f
server/backup_script.sql
```

5. Entwicklungsumgebung starten:

```
npm run dev
```

Damit starten sowohl das Next.js Frontend auf <http://localhost:3000> als auch das Backend unter <http://localhost:4000>.

Architektur und eingesetzte Technologien

Die Anwendung besteht aus einem [Next.js](#) Frontend und einem [Express](#) Backend. Als Datenbank kommt [PostgreSQL](#) zum Einsatz. Die Wahl fiel auf diese Kombination, da sie leichtgewichtig, gut erweiterbar und auch ohne großen Konfigurationsaufwand lokal ausführbar ist. Next.js liefert die React basierte Oberfläche und kann sowohl statische Seiten als auch serverseitig gerenderte

Inhalte bereitstellen. Express dient als schlanker REST-API Server, der über die `server`-Ordnerstruktur umgesetzt ist. Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über JSON-basierte HTTP-Aufrufe. Alle Benutzerdaten werden in PostgreSQL gespeichert. Für Passwort-Hashes wird `bcrypt` verwendet, Uploads werden mit `multer` direkt in der Datenbank gespeichert. In der Entwicklung laufen beide Dienste parallel über `npm run dev`.

Datenbank

Die Datenbank basiert auf PostgreSQL und wird komplett über `server/backup_script.sql` erstellt. Enthalten sind Tabellen für Benutzer, Rollen, Künstler, Touren, Events, Veranstaltungsorte sowie Tabellen zur Abwicklung von Bestellungen und zur Speicherung von Disability-Merkmalen. Das Schema ist in der folgenden Mermaid-Grafik visualisiert.

```
classDiagram
direction BT
class areas {
    varchar(50) name
    varchar(100) description
    char(3) disability_category_for
    uuid id
}
class artists {
    varchar(255) name
    text biography
    varchar(255) website
    timestamp with time zone created_at
    timestamp with time zone updated_at
    uuid artist_image
    uuid id
}
class cart_items {
    uuid cart_id
    uuid event_id
    uuid event_category_id
    integer quantity
    timestamp with time zone added_at
    boolean is_assistance_ticket
    uuid id
}
class carts {
    uuid user_id
    timestamp with time zone created_at
    timestamp with time zone updated_at
    uuid id
}
class checkout_items {
```

```
    uuid checkout_id
    uuid event_category_id
    integer quantity
    numeric(10,2) price
    timestamp with time zone added_at
    uuid event_id
    boolean is_assistance_ticket
    uuid id
}
class checkouts {
    uuid user_id
    timestamp with time zone created_at
    timestamp with time zone updated_at
    uuid id
}
class cities {
    varchar(100) name
    uuid country_id
    uuid id
}
class countries {
    varchar(100) name
    char(2) iso_code
    uuid id
}
class disability_marks {
    varchar(100) description
    uuid area_id
    char(3) mark_code
}
class event_categories {
    uuid event_id
    text name
    numeric(10,2) price
    char(3) disability_support_for
    uuid id
}
class event_supporting_acts {
    uuid event_id
    uuid artist_id
}
class event_venue_areas {
    uuid event_id
    uuid venue_area_id
    integer capacity
    uuid category_id
    uuid id
}
class events {
    uuid tour_id
    uuid venue_id
    text description
    timestamp with time zone created_at
    timestamp with time zone updated_at
    timestamp with time zone start_time
    timestamp with time zone end_time
```

```
        timestamp with time zone door_time
        uuid id
    }
    class genres {
        varchar(50) name
        uuid id
    }
    class images {
        bytea image_data
        text image_type
        text entity_type
        uuid entity_id
        uuid id
    }
    class order_tickets {
        uuid order_id
        uuid ticket_id
        uuid id
    }
    class orders {
        uuid user_id
        timestamp with time zone created_at
        varchar(255) street_address
        varchar(20) postal_code
        varchar(100) city
        varchar(100) country
        boolean is_paid
        varchar(20) salutation
        varchar(100) first_name
        varchar(100) last_name
        varchar(255) company
        uuid payment_option_id
        uuid id
    }
    class payment_options {
        varchar(50) label
        varchar(100) description
        varchar(50) icon_src
        uuid id
    }
    class shipping_options {
        varchar(100) label
        numeric price
        varchar(100) description
        uuid id
    }
    class subgenres {
        uuid genre_id
        text name
        uuid id
    }
    class tickets {
        uuid order_id
        uuid event_category_id
        varchar(50) seat_number
        numeric(10,2) price
    }
```

```
        timestamp with time zone created_at
        boolean is_assistance_ticket
        uuid id
    }
    class tour_artists {
        uuid tour_id
        uuid artist_id
    }
    class tour_genres {
        uuid tour_id
        uuid genre_id
    }
    class tour_subgenres {
        uuid tour_id
        uuid subgenre_id
    }
    class tours {
        varchar(255) title
        varchar(255) subtitle
        date start_date
        date end_date
        timestamp with time zone created_at
        timestamp with time zone updated_at
        uuid tour_image
        uuid id
    }
    class user_disability_marks {
        uuid user_id
        char(3) mark_code
    }
    class user_roles {
        varchar(50) name
        varchar(100) description
        boolean has_editing_access
        boolean has_creation_access
        boolean has_role_appointing_capability
        boolean has_account_management_access
        boolean has_disability_approval_access
        boolean has_deletion_permission
        uuid id
    }
    class users {
        varchar(20) salutation
        varchar(100) first_name
        varchar(100) last_name
        varchar(255) company
        varchar(255) street_address
        varchar(20) postal_code
        varchar(100) city
        varchar(100) country
        varchar(255) email
        varchar(20) phone
        date birth_date
        boolean request_for_disability
        integer disability_degree
        uuid disability_card_image_front
```

```

    timestamp with time zone created_at
    timestamp with time zone updated_at
    text password
    uuid disability_card_image_back
    boolean is_currently_disabled
    date disability_card_expiry_date
    uuid role
    integer visible_user_id
    uuid user_id
}
class venue_areas {
    uuid venue_id
    integer max_capacity
    uuid area_id
    uuid id
}
class venues {
    varchar(255) name
    varchar(500) address
    uuid city_id
    varchar(255) website
    timestamp with time zone created_at
    timestamp with time zone updated_at
    uuid venue_image
    uuid id
}

cart_items          --> carts          : cart_id:id
cart_items          --> event_categories : event_category_id:id
cart_items          --> events         : event_id:id
carts               --> users          : user_id
checkout_items      --> checkouts       : checkout_id:id
checkout_items      --> event_categories : event_category_id:id
checkout_items      --> events         : event_id:id
checkouts           --> users          : user_id
cities              --> countries       : country_id:id
disability_marks     --> areas          : area_id:id
event_categories     --> events         : event_id:id
event_supporting_acts --> artists       : artist_id:id
event_supporting_acts --> events       : event_id:id
event_venue_areas    --> event_categories : category_id:id
event_venue_areas    --> events         : event_id:id
event_venue_areas    --> venue_areas     : venue_area_id:id
events              --> tours          : tour_id:id
events              --> venues          : venue_id:id
order_tickets        --> orders         : order_id:id
order_tickets        --> tickets        : ticket_id:id
orders               --> payment_options : payment_option_id:id
orders               --> users          : user_id
subgenres            --> genres         : genre_id:id
tickets              --> event_categories : event_category_id:id
tickets              --> orders         : order_id:id
tour_artists         --> artists       : artist_id:id
tour_artists         --> tours         : tour_id:id
tour_genres          --> genres         : genre_id:id
tour_genres          --> tours         : tour_id:id

```



```
tour_subgenres      --> subgenres      : subgenre_id:id
tour_subgenres      --> tours          : tour_id:id
user_disability_marks--> disability_marks : mark_code
user_disability_marks--> users         : user_id
users               --> user_roles     : role:id
venue_areas         --> areas          : area_id:id
venue_areas         --> venues         : venue_id:id
venues              --> cities         : city_id:id
```

Tabellen im Detail

Nachfolgend eine kurze Beschreibung jeder Tabelle und ihrer Beziehungen.

Tabelle	Zweck / wichtige Spalten	Abhängigkeiten
countries	Länder mit ISO-Code.	–
cities	Städte innerhalb eines Landes.	<code>countries</code> via <code>country_id</code>
user_roles	Definiert Rechte einer Rolle.	–
users	Registrierte Personen samt Adresse und optionalen Disability-Angaben.	<code>user_roles</code> via <code>role</code>
artists	Künstlerinformationen.	–
genres / subgenres	Klassifikation von Touren.	<code>genres</code> via <code>genre_id</code>
tours	Übergeordnete Tour einer Reihe von Events.	–
tour_artists	Zuordnung Künstler ↔ Tour.	<code>artists</code> , <code>tours</code>
tour_genres / tour_subgenres	Genre-Verknüpfungen einer Tour.	<code>tours</code> , <code>genres / subgenres</code>
venues	Veranstaltungsorte.	<code>cities</code> via <code>city_id</code>
areas	Sitzplatz- bzw. Zugänglichkeitsbereiche.	–
venue_areas	Bereiche innerhalb eines Venues.	<code>venues</code> , <code>areas</code>
events	Konkrete Veranstaltungstermine.	<code>tours</code> , <code>venues</code>
event_categories	Ticket-Kategorien (Preis, Support für Behinderte).	<code>events</code> via <code>event_id</code>

Tabelle	Zweck / wichtige Spalten	Abhängigkeiten
event_supporting_acts	Support-Acts eines Events.	<code>events</code> , <code>artists</code>
event_venue_areas	Zuordnung Event ↔ Bereich mit Kapazität.	<code>events</code> , <code>venue_areas</code> , <code>event_categories</code>
images	Speicherung von hochgeladenen Bildern.	Beliebige Entität über <code>entity_type</code> / <code>entity_id</code>
carts	Aktiver Warenkorb eines Nutzers.	<code>users</code>
cart_items	Einzelne Artikel im Warenkorb.	<code>carts</code> , <code>events</code> , <code>event_categories</code>
checkouts	Zwischenschritt zwischen Warenkorb und Bestellung.	<code>users</code>
checkout_items	Positionen des Checkouts inkl. Preis.	<code>checkouts</code> , <code>events</code> , <code>event_categories</code>
payment_options / shipping_options	Stammdaten für Zahlungs- und Versandarten.	–
orders	Abgeschlossene Bestellungen.	<code>users</code> , <code>payment_options</code>
tickets	Konkrete Ticketdatensätze.	<code>orders</code> , <code>event_categories</code>
order_tickets	Relation zwischen Order und Ticket (1-n).	<code>orders</code> , <code>tickets</code>
disability_marks	Mögliche Merkmale auf Behindertenausweisen.	<code>areas</code> via <code>area_id</code>
user_disability_marks	Zuordnung User ↔ Marks.	<code>users</code> , <code>disability_marks</code>

Die Fremdschlüssel schützen vor inkonsistenten Daten. Viele Tabellen nutzen UUIDs als Primärschlüssel. Preise werden als `NUMERIC(10,2)` gespeichert, was zwei Nachkommastellen erlaubt. Für einfache Lookups existieren diverse Join-Tabellen (z. B. `tour_genres`).

Backend-Endpunkte

Die wichtigsten Routen des Express-Servers (siehe `server/server.js`) sind:

Methode	Pfad	Beschreibung
---------	------	--------------

Methode	Pfad	Beschreibung
POST	/register-user	Registrierung eines neuen Nutzers inkl. optionaler Behindertenausweis-Daten
POST	/login-user	Benutzeranmeldung, legt Session-Cookie an
GET	/session-status	Prüft, ob ein Nutzer eingeloggt ist und liefert Profilinformationen
POST	/logout	Beendet die aktuelle Session
GET	/user-address	Liefert die hinterlegte Anschrift des eingeloggten Nutzers
GET	/disability-marks	Auflistung möglicher Markierungen des Behindertenausweises
GET	/pending-disability-requests	Offene Anträge auf Nachteilsausgleich für den Service-Bereich
POST	/disability-requests/:id/accept	Antrag eines Nutzers akzeptieren
POST	/disability-requests/:id/decline	Antrag eines Nutzers ablehnen
GET	/artists	Auflistung aller Künstler
POST	/create-artist	Neuen Künstler anlegen
GET	/tours-detailed	Touren inkl. Events und Zugänglichkeitsdaten
POST	/create-tour	Neue Tour anlegen
GET	/venues-detailed	Liste aller Veranstaltungsorte mit Areas
POST	/create-venue	Neuen Veranstaltungsort anlegen
POST	/cart-items	Ticket zur Warenkorb-Session hinzufügen
GET	/checkout	Aktuellen Checkout laden
POST	/orders	Bestellung aus abgeschlossenem Checkout erzeugen

Dies ist nur ein Auszug. Weitere Endpunkte finden sich direkt im Quellcode von `server/server.js`.

Seitenübersicht

Die Next.js Anwendung befindet sich unter `src/pages` und nutzt dynamische Routen. Wichtige Seiten sind:

Pfad	Zweck / Inhalte
/	Startseite mit Highlights sowie Künstler- und Tourübersicht
/artists/[artist]	Detailseite eines Künstlers und Übersicht zugehöriger Touren
/artists/[artist]/[tour]	Informationen und Events einer Tour
/artists/[artist]/[tour]/[event]	Detailseite eines Events mit Buchungsoption
/registration	Formular zur Kontoerstellung und Erfassung optionaler Behindertenausweis-Daten
/login	Anmelden bzw. Schnellanmeldung inkl. Passwort-Reset
/profile	Persönliche Daten, Bestellungen und eigene Events verwalten
/checkout	Mehrstufiger Bestellprozess (Versanddaten, Zahlung, Abschluss)
/admin	Einstieg in alle Admin-Unterseiten zur Pflege von Künstlern, Ländern, Genres, Touren und Veranstaltungsorten
/service	Zugriffspunkt für Service-Mitarbeiter (u.a. Nachteilsausgleichsanträge und Account-Management)

Alle Seiten unter `/admin/*` und `/service/*` setzen entsprechende Berechtigungen voraus.

Rollen

Es existieren drei grundlegende Benutzerrollen:

Rolle	Berechtigungen
User	Kann Tickets kaufen, eigenes Profil bearbeiten und Nachteilsausgleich beantragen.
Service	Darf Service-Seiten nutzen, z.B. Anträge prüfen oder Accounts verwalten.
Admin	Umfasst sämtliche Rechte: Erstellung/Bearbeitung von Daten, Rollenvergabe und Zugriff auf alle Admin-Seiten.

Die Zuordnung der Rechte erfolgt über die Tabelle `user_roles` . Beim Registrieren erhält jeder Nutzer standardmäßig die Rolle **User**.

Komponenten und Hooks

Der Quellcode unter `src` gliedert sich in wiederverwendbare React-Komponenten und mehrere Custom Hooks:

Komponenten (Auswahl)

Datei	Einsatz
<code>nav-bar.jsx</code>	Hauptnavigation mit Suche und Warenkorb-Anzeige
<code>footer.jsx</code>	Gemeinsamer Seitenfuß
<code>LoadingOverlay.jsx</code>	Überblendet die Seite während Daten geladen werden
<code>DisabilityRequestModal.jsx</code>	Dialog zur Beantragung eines Nachteilsausgleichs
<code>DeleteAccountModal.jsx</code>	Bestätigungsdialog zum Löschen des Accounts
<code>smallArtistCard.jsx</code> / <code>smallTourCard.jsx</code>	Vorschaukarten auf der Startseite

Hooks

Hook	Zweck
<code>useAuth</code>	Kümmert sich um Login-Status und hält Userdaten clientseitig vor
<code>useCart</code>	Verwaltet den Warenkorb und lädt Items vom Server
<code>useRequireAccess</code>	Leitet unberechtigte Nutzer auf die Login-Seite um
<code>useValidation</code>	Hilft bei Formularvalidierungen

Die Komponenten sind so aufgebaut, dass sie in verschiedenen Seiten wiederverwendet werden können. Die Hooks stellen gemeinsam genutzte Logik bereit und erleichtern die Anbindung an das Backend.

Weiterführende Hinweise

- Das Frontend erwartet als API-Basis `NEXT_PUBLIC_API_URL` (Standard: `http://localhost:4000`).
- Für Datei-Uploads wird `multer` verwendet. Bilder werden in der Tabelle `images` gespeichert und über `/image/:id` ausgeliefert.
- Ein Cronjob im Server entfernt veraltete Warenkörbe und Checkouts alle 60 Sekunden.

Projektstruktur

Das Repository besteht aus zwei Hauptebenen:

- **eventim-disability-integration** – enthält die Next.js Anwendung inkl. Express-Server unter `server/`.
- **README.md** und Konfigurationsdateien auf Root-Ebene dokumentieren das Gesamtprojekt.

Wichtige Verzeichnisse innerhalb der App:

- `server` – Backend mit REST-Endpunkten und Datenbankanbindung.
- `src/pages` – Seiten des Frontends, vielfach mit dynamischen Routen.
- `src/components` – Wiederverwendbare React-Komponenten.
- `src/hooks` – Custom Hooks für Authentifizierung, Warenkorb usw.

Security & Fehlertoleranz

Datenverschlüsselung und Zugriffsrechte

- Passwörter werden mit `bcrypt` gehasht und niemals im Klartext gespeichert.
- Die Session verwendet ein Secret aus `credentials.json` und legt ein HTTP-only Cookie ab.
- CORS ist auf die eigene Origin beschränkt; Uploads erfolgen über `multer` direkt in die Datenbank.

Rollen und Fähigkeiten

Die Tabelle `user_roles` definiert Berechtigungen. Aktuell existieren drei Rollen:

Rollenname	Beschreibung	Edit	Create	Appoint Roles	Account Mgmt	Disability Approval	Delete
user	Regulärer Eventim Nutzer	false	false	false	false	false	false

Rollenname	Beschreibung	Edit	Create	Appoint Roles	Account Mgmt	Disability Approval	Delete
service	Service-Mitarbeiter von Eventim	false	false	false	true	true	false
admin	Vollzugriff auf alle Funktionen	true	true	true	true	true	true

Fehlertoleranz des Backends

- `server/db.js` überwacht die Datenbankverbindung und versucht bei Fehlern einen Reconnect. Ein Circuit-Breaker auf Basis von `opossum` verhindert Kaskadenfehler.
- `server/server.js` startet erst, wenn eine DB-Verbindung besteht, und führt regelmäßige Aufräumjobs aus.
- Der Server wird mit **PM2** im Cluster-Modus betrieben und startet bei Fehlern automatisch neu.
- Ungefangene Fehler führen zu einem kontrollierten Exit, damit PM2 unmittelbar einen Neustart durchführen kann.

Potenzielle Verbesserungen

- HTTPS/TLS für alle Verbindungen erzwingen.
- Stärkere Validierung der Eingaben und Rate-Limiting für API-Calls.
- Zentrales Logging (ELK-Stack o.Ä.) für die PM2-Prozesse einbinden.

Nächste Schritte

Das Grundgerüst funktioniert lokal stabil: Registrierung, Login, Rollenverwaltung und der Bestellprozess sind lauffähig. Das Projekt ist in Frontend und Backend klar getrennt und setzt auf PostgreSQL als persistente Datenbasis.

Vor einer finalen Bereitstellung sollten noch folgende Punkte bearbeitet werden:

1. **Konfiguration bereinigen** – Secrets in Umgebungsvariablen auslagern und Produktionsbuilds automatisieren.
2. **Security prüfen** – HTTPS erzwingen, CORS-Regeln präzisieren und optionale 2FA vorsehen.
3. **Fehlertoleranz weiter ausbauen** – zentrales Logging und Alerting einrichten.

Bereits sehr gut funktionieren das Rollenmodell, die Bild-Uploads sowie die periodische Datenbereinigung im Backend.

